

Évaluer des arbres de décision

Surapprentissage, matrices de confusion et limites des classificateurs

Traduit et adapté depuis Bootstrap World (Fall 2026)

Suite de la leçon « Construire des arbres de décision »

Licence Creative Commons 4.0 International

*Ces supports ont été développés en partie grâce au soutien de la National Science Foundation
(bourses 1042210, 1535276, 1648684, 1738598, 2031479 et 1501927).*

Table des matières

1	Construire des arbres de décision en Pyret	3
1.1	Lancement	3
1.2	Exploration : <code>build-tree</code>	3
1.2.1	Construire des arbres à un niveau	4
1.2.2	Construire des arbres à plusieurs niveaux	5
1.3	Comparer les arbres	6
1.4	Utiliser nos classificateurs avec le jeu de test	7
1.5	Synthèse	8
2	Évaluer un classificateur	9
2.1	Lancement : tests médicaux et classificateurs	9
2.2	Construire une matrice de confusion à la main	11
2.3	Laisser Pyret construire la matrice	12
2.3.1	Lire une matrice de confusion	13
2.4	Synthèse	13

Construire des arbres de décision en Pyret

Objectif de la section

Maintenant que les élèves connaissent l'algorithme de construction d'un arbre de décision, iels explorent la capacité de Pyret à construire des arbres *automatiquement*. Iels étudient l'impact de la profondeur maximale et du choix des colonnes sur la qualité d'un classificateur.

Lancement

Récapitulons ce que nous savons sur les classificateurs :

- Il existe un algorithme simple pour calculer un *arbre optimal* pour n'importe quel jeu de données.
- Les arbres de décision sont en réalité des fonctions appelées *classificateurs* : iels prennent une entrée et produisent une prédiction.
- Tous les arbres de décision ne se valent pas !

Bien sûr, calculer un arbre de décision à la main prend beaucoup de temps, surtout s'il y a des dizaines de milliers de lignes et des centaines de colonnes ! Heureusement, les ordinateurs sont *vraiment, vraiment* doués pour suivre des algorithmes.

Point clé

Un exemple d'application précoce de ce type d'automatisation est utilisé par la **United States Postal Service** (service postal des États-Unis) pour lire les adresses manuscrites sur les enveloppes. Leur système d'interprétation d'adresses a été entraîné sur la base de données **MNIST** (*Modified National Institute of Standards and Technology*), qui contient des milliers d'images étiquetées de chiffres manuscrits.

Exploration : **build-tree**

Étant donné un jeu de données sur lequel s'entraîner, Pyret peut construire des arbres de décision *optimaux* pour nous, et même nous donner le code du classificateur correspondant — en utilisant exactement l'algorithme que vous connaissez déjà !

```
# build-tree :: (Table t, List<String> colonnes, String  
colonne-reponse, Number profondeur-max) -> DecisionTree
```

La fonction `build-tree` prend **quatre arguments** dans son Domaine :

1. Le *jeu de données d'entraînement* sur lequel ajuster l'arbre.
2. Une *liste de noms de colonnes* sur lesquelles les nœuds peuvent se diviser.
3. Le nom de la *colonne cible* contenant les « réponses » (les catégories à prédire).
4. La *profondeur maximale* autorisée pour l'arbre.

Construire des arbres à un niveau

Activité

Dans la Zone de Définitions, définissez `tree-c1` comme un arbre à 1 niveau utilisant uniquement la colonne catégorielle "queue", puis cliquez sur « Run » :

```
tree-c1 = build-tree(training, [list: "queue"], "espece", 1)
```

1. Évaluez `tree-c1`. Si un animal n'a pas de queue, que prédit-il?
2. Évaluez `dt-code(tree-c1)`. Où avez-vous déjà vu ce code?
 - C'est la même fonction que `classifieur-queue`, déjà présente dans notre Zone de Définitions!

Définissez maintenant `tree-q1` comme un arbre à 1 niveau utilisant uniquement la colonne quantitative "poids", puis cliquez sur « Run » :

```
tree-q1 = build-tree(training, [list: "poids"], "espece", 1)
```

3. Évaluez `tree-q1`. Quelle division Pyret a-t-il trouvée?
 - Il a trouvé qu'il était plus pertinent de diviser à **9,55 kg**!

Définissez `tree-any1` comme un arbre à 1 niveau autorisé à utiliser *n'importe quelle* colonne :

```
tree-any1 = build-tree(training, [list: "sexe", "queue", "mammifere", "nage", "poids"], "espece", 1)
```

4. Quelle colonne *auriez-vous* choisie pour le nœud racine? Évaluez `tree-any1`. Quelle colonne *Pyret* a-t-il choisie?
 - "nage" — c'est la colonne qui sépare le mieux les données.

Construire des arbres à plusieurs niveaux

Activité

Ajoutez les définitions suivantes à la Zone de Définitions et cliquez sur « Run » :

```
tree-c2 = build-tree(training, [list: "queue"], "espece", 2)
tree-q2 = build-tree(training, [list: "poids"], "espece", 2)
tree-any2 = build-tree(training, [list: "sexe", "queue", "mammifere", "nage", "poids"], "espece", 2)
```

1. Dans le code ci-dessus, nous avons autorisé `tree-c2` à avoir une profondeur maximale de 2. L'arbre résultant n'a en fait qu'un niveau. Pourquoi ?
 - Une fois qu'on a divisé sur `queue`, il n'y a plus rien à trier avec *seulement* la colonne `"queue"`.
2. En quoi les arbres `tree-c1` et `tree-c2` sont-ils différents ? Identiques ?
 - Ils sont identiques ! La colonne `queue` ne permet qu'une seule division utile.

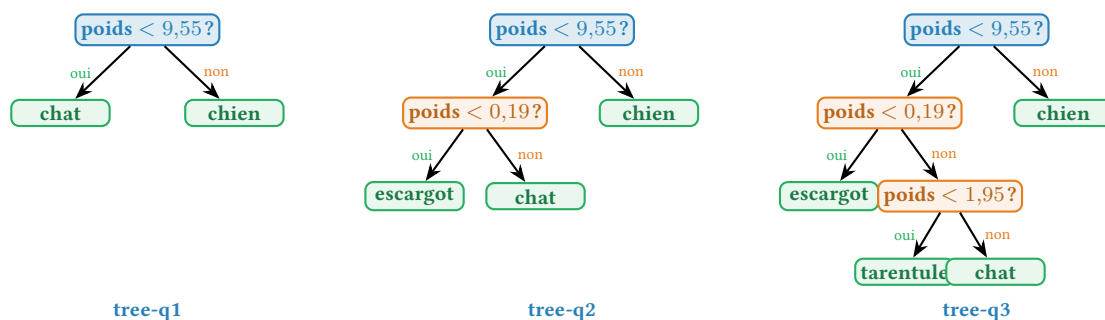
Ajoutez maintenant :

```
tree-q3 = build-tree(training, [list: "poids"], "espece", 3)
tree-q4 = build-tree(training, [list: "poids"], "espece", 4)
tree-q5 = build-tree(training, [list: "poids"], "espece", 5)
tree-any3 = build-tree(training, [list: "sexe", "queue", "mammifere", "nage", "poids"], "espece", 3)
tree-any4 = build-tree(training, [list: "sexe", "queue", "mammifere", "nage", "poids"], "espece", 4)
tree-any5 = build-tree(training, [list: "sexe", "queue", "mammifere", "nage", "poids"], "espece", 5)
```

3. Comparez `tree-q1`, `tree-q2` et `tree-q3`. En quoi sont-ils similaires ? Différents ?
 - Ils commencent tous par *la même division* (`poids < 9,55`), mais gagnent en profondeur pour mieux séparer les espèces.

Code Pyret

Les trois arbres `tree-q1`, `tree-q2` et `tree-q3` construits uniquement sur la colonne `"poids"` :



Comparer les arbres

Activité

Assurez-vous d'avoir enregistré toutes les nouvelles définitions dans votre copie du fichier `Arbre-Decision-Starter.arr`. Évaluez chaque arbre et utilisez la fonction `classify` pour compter combien d'animaux du jeu d'entraînement sont correctement classifiés.

arbre	niv. max	niv. réels	# correct (entraînement)	# correct (test)
tree-c1	1	1		
tree-q1	1	1		
tree-any1	1	1		
tree-c2	2			
tree-q2	2			
tree-any2	2			
tree-q3	3			
tree-any3	3			
tree-q4	4			
tree-any4	4			
tree-q5	5			
tree-any5	5			
tree-q6	6			

Notez que `tree-q6` fonctionne aussi bien que `tree-any4`, même s'il n'a qu'une seule colonne pour s'entraîner! Comme c'est souvent le cas pour les colonnes quantitatives, il y a davantage de variabilité dans `poids` que dans n'importe laquelle de nos colonnes catégorielles. En fait, le seul moyen de ne *pas* avoir un classificateur parfait basé sur `poids` serait que deux animaux d'espèces différentes pèsent exactement le même poids!

Activité

1. Que remarquez-vous? Que vous demandez-vous?
2. Autoriser davantage de niveaux rendait-il *toujours* l'arbre meilleur?
3. Si nous utilisons une seule colonne, laquelle ferait le meilleur travail pour construire un arbre sur les données d'entraînement : catégorielle ou quantitative?

Utiliser nos classificateurs avec le jeu de test

Activité

Nous avons construit nos classificateurs à partir du jeu de données `training`. Voyons comment iels s'en sortent pour prédire l'"`espece`" d'un nouveau jeu de données appelé `testing`. Classifiez chaque arbre de décision sur `testing`, par exemple :

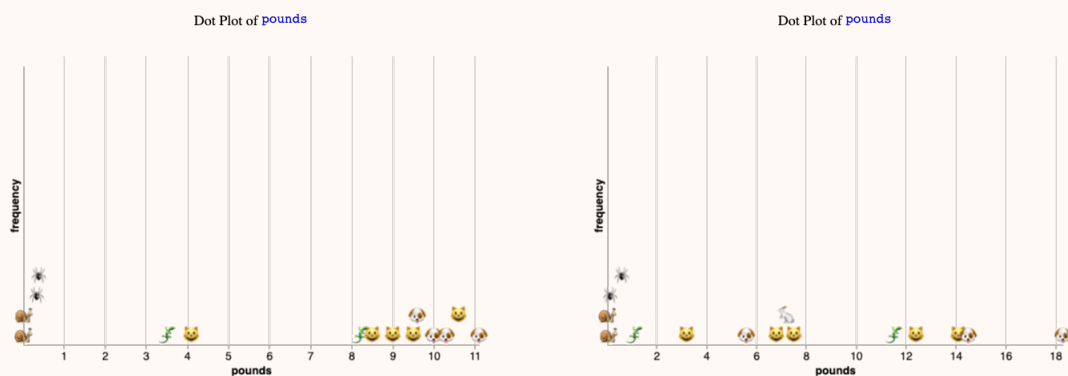
```
classify(testing, "espece", tree-c1)
```

1. Quels deux modèles ont fait le meilleur travail pour prédire l'espèce sur le jeu d'entraînement ?
2. Comment leurs taux de réussite se comparent-ils sur le jeu de *test* ?
3. Regardez attentivement le nombre de prédictions correctes pour les arbres `tree-q*` construits sur la colonne `poids`, à mesure que le nombre de niveaux autorisés augmente. Que remarquez-vous ?

Réflexion

Pourquoi `tree-q6` a-t-il si mal performé sur le jeu de test ?

Laissez les élèves partager leurs théories.



Dot plot du `poids` sur le jeu d'entraînement (gauche) et le jeu de test (droite). Chaque point est un emoji de l'espèce.

Point clé**Surapprentissage** (*over-fitting*)

C'est un exemple de ce que les ingénieur·es en apprentissage automatique appellent le **surapprentissage**. Plus un classificateur dépend des particularités d'un attribut dans les données d'entraînement, plus il est sensible aux variations de cet attribut quand vient le moment d'utiliser de nouvelles entrées. Ici, `tree-q6` était *totalement dépendant de seuils très spécifiques dans la colonne poids*, donc quand le jeu de test a eu des seuils différents, le classificateur a performé mal.

Ce serait comme réviser pour un examen en mémorisant les réponses à un examen précédent. Vous avez appris à réussir *ce test précis*, mais pas assez pour réussir un autre examen.

En apprentissage automatique, le **surapprentissage** se produit quand notre arbre de décision essaie si fort de trouver un motif dans un ensemble spécifique de données qu'il commence à « mémoriser » les détails et le bruit aléatoire au lieu d'apprendre la règle réelle. C'est exactement ce qui s'est passé avec la colonne `poids` ici.

Les autres colonnes ne variaient pas autant! Les chiens et les chats sont tous des mammifères quoi qu'il arrive. La plupart des chiens aiment nager. Et la plupart des chiens, chats et lézards ont une queue. Ces attributs sont plus stables entre les échantillons, alors que `poids` peut varier largement.

Synthèse

Réflexion

- Expliquez comment la table du jeu de données d'entraînement est liée à l'arbre de décision.
 - Les en-têtes de colonnes de la table sont les questions qui seront posées dans les nœuds de décision.
 - Les données dans chaque colonne sont les réponses à ces questions, qui deviennent les étiquettes des arêtes de l'arbre (« oui » et « non » pour ce jeu de données).
 - Les catégories de sortie de la table sont les nœuds feuilles de l'arbre.
- Un arbre de décision a-t-il généralement le même nombre de nœuds feuilles qu'il y a de lignes dans le jeu de données d'entraînement? Pourquoi?
 - Non. L'algorithme de division fonctionne *par attribut*; en général, plusieurs lignes d'un jeu d'entraînement partagent des attributs. Donc, à moins que chaque ligne ait une combinaison unique d'attributs *et* que l'arbre puisse aller infiniment profond, l'arbre aura moins de feuilles qu'il n'y a de lignes.
- Un arbre de décision peut-il avoir *moins* de nœuds feuilles qu'il n'y a de catégories uniques dans le jeu de données d'entraînement? Pourquoi?
 - Oui. Un arbre peut ne pas être assez profond pour classer correctement chaque catégorie — et dans certains cas, il peut ne *pas y avoir* de façon de classer correctement chaque catégorie!

Évaluer un classificateur

Objectif de la section

Les élèves découvrent les **matrices de confusion** et utilisent Pyret pour les générer à partir des fonctions classificateurs dérivées des arbres de décision.

Lancement : tests médicaux et classificateurs

Les tests pour la grippe, la rougeole, etc. sont toutes des formes de classificateurs ! En examinant chacun de ces tests, les responsable-es de santé publique doivent penser aux cas suivants :

Statut réel	Prédit positif	Prédit négatif
Infecté	Vrai positif : le test dit infecté, et la personne l'est <i>réellement</i>	Faux négatif : le test dit infecté, mais la personne ne l'est pas
Non infecté	Faux positif : le test dit sain, mais la personne est malade	Vrai négatif : le test dit sain, et la personne l'est <i>réellement</i>

Point clé

Il y a deux façons pour le test d'avoir *raison*, et deux façons d'avoir *tort*. Supposons que nous ayons une épidémie d'une maladie vraiment mortelle et hautement contagieuse, et deux tests possibles au choix :

- **Test A** produit beaucoup de *faux positifs* : il fait presque toujours la bonne chose si vous êtes malade, mais beaucoup de personnes saines se font dire qu'elles sont infectées.
- **Test B** produit beaucoup de *faux négatifs* : il fait presque toujours la bonne chose si vous êtes sain-e, mais beaucoup de personnes malades se font dire qu'elles vont bien.

Réflexion

Vous êtes ministre de la santé d'un pays qui subit une épidémie de cette maladie.

- Quel test choisissez-vous pour votre pays ?
- Choisiriez-vous un test différent pour une maladie *moins* contagieuse ?

Laissez aux élèves le temps de discuter. Ce qui compte n'est pas que la classe aboutisse à une conclusion, mais que les élèves commencent à voir qu'il ne s'agit pas simplement de « quelle est la précision du test ? », mais plutôt « dans quels cas le test se trompe-t-il, et combien de fois ? ».

Point clé

Cet exemple concret montre clairement qu'il ne suffit pas de dire « le classificateur avait raison la moitié du temps ».

Sur les 5 chats du jeu de données, combien notre `classifieur-queue` a-t-il classifiés *correctement* ?

- **5**. Soit $\approx 100\%$.

Sur les 2 tarentules, combien notre `classifieur-queue` a-t-il classifiées *correctement* ?

- **0**. Soit $\approx 0\%$.

Les rapports que les data scientist-es (et les responsable-es de santé publique !) utilisent pour comprendre à quel point un classificateur est « bon » s'appellent des **matrices de confusion**. Une matrice de confusion montre généralement des *ratios* :

- 0 sur 4 chiens devient 0 % (soit 0 en décimal)
- 2 sur 2 lézards devient 100 % (soit 1)
- 4 sur 5 chats devient 80 % (soit 0,8)

... et ainsi de suite.

Construire une matrice de confusion à la main

Activité

Ajoutez ce classificateur à votre Zone de Définitions et cliquez sur « Run » :

```
fun classifieur-nage(r):
  if r["nage"] == true:
    "chien"
  else:
    "chat"
  end
end
```

Avec votre voisin·e, utilisez la fonction `classify` pour remplir la matrice de confusion ci-dessous pour `classifieur-nage` sur le jeu `training` :

- Remplissez d’abord le « sur X » (le nombre d’animaux de cette espèce dans le jeu).
- Convertissez ensuite le compte de prédictions correctes en *pourcentage*.
- Enfin, convertissez le pourcentage en *décimal*.

espece réelle \ prédit	chat	chien	lizard	escargot	tarentule
"chat"	4 sur 5 = 80 % = 0,8	sur 5	sur 5	sur 5	sur 5
"chien"	sur 4	sur 4	sur 4	sur 4	sur 4
"lizard"	sur 2	sur 2	sur 2	sur 2	sur 2
"escargot"	sur 2	sur 2	sur 2	sur 2	sur 2
"tarentule"	sur 2	sur 2	sur 2	sur 2	sur 2

actual-species	predicted-cat	predicted-dog	predicted-lizard	predicted-snail	predicted-tarantula
"cat"	0.8	0.2	0	0	0
"dog"	0	1	0	0	0
"lizard"	0.5	0.5	0	0	0
"snail"	1	0	0	0	0
"tarantula"	1	0	0	0	0

FIGURE 1 – Capture d’écran d’une matrice de confusion Pyret pour le `classifieur-nage`, évaluée sur le jeu de données d’entraînement.

Réflexion

- Quelle espèce `classifieur-nage` a-t-il prédite le *plus* précisément ?
 - Les chiens.
- Quelle espèce a-t-il prédite le *moins* précisément ?
 - Les escargots, les lézards et les tarentules.
- Qu'apprend-on d'autre de la matrice de confusion ?
 - Tous les escargots et tarentules sont mal classifiés en chats.
 - La moitié des lézards sont mal classifiés en chats, l'autre moitié en chiens.
 - 20 % des chats sont mal classifiés en chiens.

Laisser Pyret construire la matrice

Pyret possède une fonction qui construit une matrice de confusion pour vous ! Voici le contrat :

```
# confusion-matrix :: (Table t, String colonne-reponse,  
  (Row -> String) classifieur) -> Table
```

Activité

Testez la matrice de confusion pour notre `classifieur-nage` avec le code suivant :

```
confusion-matrix(training, "espece", classifieur-nage)
```

1. Comparez la matrice que vous avez faite à la main avec celle produite par Pyret. Si elles ne correspondent pas, travaillez avec votre voisin·e pour comprendre pourquoi.
2. Générez une matrice de confusion pour `classifieur-nage` sur la table `testing`. Que remarquez-vous ? Comment la performance sur `testing` se compare-t-elle à celle sur `training` ?
 - Iel s'en sort très bien, mais mal classifie tous les lapins en chiens.
 - On ne s'attendrait pas à ce qu'il classifie correctement l'espèce d'un animal sur lequel il ne s'est pas entraîné.
3. Construisez une matrice de confusion pour notre `classifieur-queue` original et préparez-vous à partager ce que vous apprenez.

Lire une matrice de confusion

Point clé

Voici la matrice de confusion pour `classifieur-queue` sur le jeu d'entraînement :

espece réelle \ prédit	chat	chien	lézard	escargot	tarentule
"chat"	1	0	0	0	0
"chien"	1	0	0	0	0
"lézard"	1	0	0	0	0
"escargot"	0	0	0	1	0
"tarentule"	0	0	0	1	0

Les « bons scores » apparaissent sur la **diagonale**, du coin supérieur gauche au coin inférieur droit. **Note : nous utilisons des valeurs entre 0 et 1, plutôt que des pourcentages!** « 80 % » apparaîtrait comme 0.8.

Réflexion

- Que pouvons-nous déduire de cette matrice de confusion ?
 - Le classificateur a 100 % des chats et des escargots corrects.
 - Iel a mal classifié tous les chiens en chats.
 - Iel a mal classifié tous les lézards en chats.
 - Iel a mal classifié toutes les tarentules en escargots.
- À quoi ressemblerait la matrice pour le **meilleur classificateur possible** ?
 - Des 1 sur la diagonale, montrant :
 - 100 % des chats prédits en chats
 - 100 % des chiens prédits en chiens
 - 100 % des lézards prédits en lézards
 - ... etc.
 - Des 0 partout ailleurs, montrant qu'il n'y a pas de mauvaises classifications.

Synthèse

Réflexion

- Pourquoi un arbre de décision précis à 100 % sur les données d'entraînement ne fait-il pas toujours de bonnes prédictions sur les données de test ?
 - Si le classificateur est surajusté (*overfit*) aux données d'entraînement, il pourrait ne pas faire de bonnes prédictions sur des données liées mais différentes.
 - Les modèles basés sur des colonnes quantitatives sont particulièrement sensibles au surapprentissage.
- Avec vos propres mots, décrivez ce qu'est une matrice de confusion.

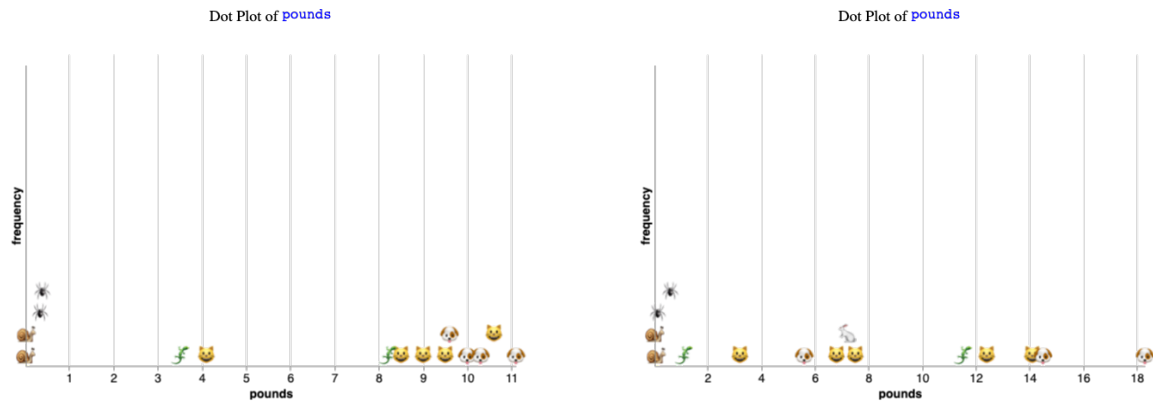
Point clé

Tous les modèles se trompent. Certains sont utiles.

- Quand un arbre de décision se trompe-t-il ?
- Quand est-il utile ?

Annexe A : Visualiser le surapprentissage

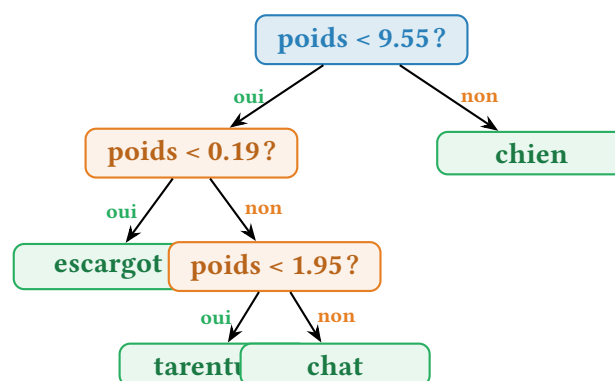
Les *dot plots* ci-dessous comparent la distribution du poids entre le jeu d'entraînement et le jeu de test. Chaque point est un emoji de l'espèce de l'animal : chat, chien, lézard, escargot, tarentule, et (dans le jeu de test) lapin. Les emojis exacts sont visibles sur les dot plots ci-dessous.



Gauche : jeu d'entraînement (chats, chiens, lézards, escargots, tarentules). Droite : jeu de test (mêmes espèces + lapins).

Pourquoi **tree-q6** se trompe-t-il sur le jeu de test ?

L'arbre **tree-q6** a été construit en utilisant *uniquement* la colonne **poids** avec une profondeur maximale de 6 niveaux. Sur le jeu d'entraînement, il a trouvé des seuils *parfaits* qui séparent chaque espèce. Mais ces seuils sont très spécifiques : par exemple, « **poids** < 9,55 » sépare les chats des chiens dans *ce* jeu d'entraînement, mais un chat plus lourde qu'un chien dans le jeu de test sera mal classifié.



L'arbre **tree-q3** ci-dessus est parfait sur le jeu d'entraînement, mais se trompe sur le jeu de test dès qu'un animal a un poids inattendu. C'est le surapprentissage.

Annexe B : Fichiers du projet

Fichier	Description
Arbre-Decision-Starter.arr	Fichier de démarrage Pyret (inchangé par rapport à la leçon précédente). Fournit <code>build-tree</code> , <code>confusion-matrix</code> , <code>classify</code> , <code>dt-code</code> , etc. via <code>ai-library.arr</code> .
<code>evaluer-arbres-decision.tex</code>	Document de cours (ce fichier)
<code>images/320f755b0fe617c5.png</code>	Dot plot du poids, jeu d'entraînement
<code>images/c321846ad89322dc.png</code>	Dot plot du poids, jeu de test (avec lapins)
<code>images/b74bc727477a4c35.png</code>	Capture d'écran d'une matrice de confusion Pyret
<code>images/CCbadge.png</code>	Logo Creative Commons

À noter : cette leçon utilise *le même* fichier `Arbre-Decision-Starter.arr` que la leçon précédente (Construire des arbres de décision). Aucune modification du code n'est nécessaire : `build-tree`, `confusion-matrix`, `dt-code`, `classify` sont fournis par la bibliothèque `ai-library.arr` de Bootstrap, chargée via `use context url-file(...)`.

Annexe C : Pour aller plus loin

Notes pour l'enseignant·e

Bientôt disponible!

Vous voulez que vos élèves pratiquent leurs compétences et appliquent leurs connaissances des classificateurs à des données du monde réel? Faites-leur ouvrir le *AI Music Starter File* de Bootstrap (<https://pyret.BootstrapWorld.org/editor#shareurl=https://raw.githubusercontent.com/bootstrapworld/starter-files/refs/heads/main/ai/ai-music.arr>) et construire un arbre de décision capable de différencier les chansons R&B, Rock'n'Roll et Country! Le fichier de démarrage est prêt aujourd'hui; nous mettrons à jour la page du cahier d'exercices et ajouterons une section optionnelle à cette leçon qui structure l'ensemble.

À propos de ces supports

Ces supports ont été développés en partie grâce au soutien de la *National Science Foundation* (bourses 1042210, 1535276, 1648684, 1738598, 2031479 et 1501927).

Bootstrap par la Communauté Bootstrap est sous licence Creative Commons 4.0 Unported. Cette licence n'accorde pas la permission d'organiser des formations ou du développement professionnel.

Traduction et adaptation française réalisées en juillet 2026.

Source originale : <https://www.bootstrapworld.org/materials/fall2026/en-us/lessons/decision-trees-evaluating/>